Minor Assignment 2 - CS771

Golla Lekha Harshaa
CSE / 240402
lekhahg24@iitk.ac.in

Korada Jayavardhan
CSE / 240554
jkorada24@iitk.ac.in

Yash Raj
CSE / 241202
yashr24@iitk.ac.in

Panchaneni Ashwin Rao
CSE / 240727
ashwinr24@iitk.ac.in

Vandana
Statistics / 252080612
vandana25@iitk.ac.in

Problem 2.1: Melbo Mends Missing Images

Part 1: Joint Optimization (Single-Step Inference)

Aim: We want to do a joint optimization to find both the best class prediction ($\hat{y}$) and the best reconstruction for the missing pixels ($\hat{x}^m$) at the same time. We need to solve:

$$\{\hat{y}, \hat{x}^m\} = \arg\max_{c,v} \mathbb{P}[y = c, x^m = v | x^o]$$

Using the product rule of probability, we can split this joint probability into two parts:

$$\mathbb{P}[y = c, x^m = v | x^o] = \mathbb{P}[x^m = v | y = c, x^o] \cdot \mathbb{P}[y = c | x^o]$$

For the second part ($\mathbb{P}[y = c|x^o]$), we can use Bayes' rule. It is proportional to the likelihood of the observed pixels times the prior probability of the class:

$$\mathbb{P}[y = c|x^o] \propto \mathbb{P}[x^o|y = c] \cdot \mathbb{P}[y = c]$$

And, replacing that back in, our main goal is to maximize this whole expression over c and v:

$$\arg\max_{c,v} (\mathbb{P}[x^m = v|y = c, x^o] \cdot \mathbb{P}[x^o|y = c] \cdot \mathbb{P}[y = c])$$

If we just look at maximizing for v (the missing pixels) right now, the only term in our equation that actually depends on v is the first one: $\mathbb{P}[x^m = v|y = c, x^o]$.

From the assignment, we know that because the original classes are modeled as single Gaussians, the conditional probability of the missing pixels is also a Gaussian distribution:

$$x^m | x^o, y = c \sim \mathcal{N}(\bar{\mu}^c, \bar{\Sigma}^c)$$

Where the parameters are:

$$\bar{\mu}^c = \mu_c^m + \Sigma_{mo}^c (\Sigma_{oo}^c)^{-1} (x^o - \mu_c^o)$$
$$\bar{\Sigma}^c = \Sigma_{mm}^c - \Sigma_{mo}^c (\Sigma_{oo}^c)^{-1} \Sigma_{om}^c$$

Since it is a Gaussian, the highest possible probability density is always exactly at the mean. So, for any given class c, the best guess for the missing pixels is just:

$$v_c^* = \bar{\mu}^c$$

Now, to calculate the actual probability value at that peak, if we substitute the mean into the Gaussian formula, the $e^{\cdots}$ part just becomes $e^0 = 1$. We are just left with the normalization constant in the front:

$$\max_v \mathbb{P}[x^m = v|y = c, x^o] = \frac{1}{\sqrt{(2\pi)^k |\bar{\Sigma}^c|}}$$

Preprint.

(where k is just the number of missing pixels).

Now we take that peak value we just found and substitute it back into our main equation from Step 1 to solve for the class $\hat{y}$:

$$\hat{y} = \arg\max_c \left(\frac{1}{\sqrt{(2\pi)^k |\bar{\Sigma}^c|}} \cdot \mathbb{P}[x^o | y = c] \cdot \mathbb{P}[y = c] \right)$$

Since $(2\pi)^k$ is just a constant that doesn't change depending on the class c, we can just ignore it for the argmax. This leaves us with the final single-step prediction rule:

$$\hat{y} = \arg\max_c \left(\frac{1}{\sqrt{|\bar{\Sigma}^c|}} \cdot \mathbb{P}[x^o | y = c] \cdot \mathbb{P}[y = c] \right)$$

Are the single-step and two-step inference procedures identical? No, they are not identical.

If you look at the lecture code, the two-step method just predicts the class by maximizing the observed likelihood and the prior:

$$\hat{y}_{\text{two-step}} = \arg\max_c \left(\mathbb{P}[x^o | y = c] \cdot \mathbb{P}[y = c] \right)$$

But our new single-step method has an extra term squeezed in there: $\frac{1}{\sqrt{|\bar{\Sigma}^c|}}$.

Now, because every class has its own specific covariance matrix, the determinant of the conditional covariance ($|\bar{\Sigma}^c|$) will be different for each class. This extra term acts like a penalty for uncertainty. If a class model is really "unsure" about what the missing pixels should look like (meaning it has a high variance/spread for those pixels), its probability score gets lowered.

Because of this penalty term, it is completely possible for the two-step method and the single-step method to predict two different digits for the exact same image.

Part 2: Code Modifications

To implement the modified single-step technique, I did not change the training code. The function `learnClassConditionalDist(...)` remains unchanged, so each class is still modeled by a single Gaussian with the same empirical mean and covariance as in the lecture notebook.

The required changes are only in the test-time inference code.

1. I kept the original lecture functions `predictGen(...)` and `reconstructImages(...)` unchanged so that they can still be used as the two-step baseline in Part 3.
2. I added a precomputation step for the modified method. For each class c, I extract the observed and missing blocks of the mean and covariance, compute the pseudo-inverse of $\Sigma_{oo}^{(c)}$, compute the conditional covariance

   $$\bar{\Sigma}_c = \Sigma_{mm}^{(c)} - \Sigma_{mo}^{(c)}(\Sigma_{oo}^{(c)})^{\dagger}\Sigma_{om}^{(c)},$$

   and store the matrix

   $$B_c = (\Sigma_{oo}^{(c)})^{\dagger}\Sigma_{om}^{(c)}.$$

   I also store the class-dependent constant

   $$\alpha_c = \log p(y=c) - \frac{1}{2}\log\det(\Sigma_{oo}^{(c)}) - \frac{1}{2}\log\det(\bar{\Sigma}_c).$$

3. The reason $\log\det(\Sigma_{oo}^{(c)})$ appears above is that the observed pixels are modeled by the Gaussian density $p(x_o \mid y=c)$. In log form, this contributes both a log-determinant term and a quadratic Mahalanobis term. Hence the class score is written as

   $$\text{score}_c(x_o) = \alpha_c - \frac{1}{2}(x_o - \mu_o^{(c)})^{\top}(\Sigma_{oo}^{(c)})^{\dagger}(x_o - \mu_o^{(c)}),$$

   where α_c stores only the class-dependent terms, while the quadratic term is computed separately for each test image.
4. I then added a new function `predictJointAndReconFast(...)`. This function first computes the joint score for every class, predicts the label

   $$\hat{y} = \arg\max_c \text{score}_c(x_o),$$

   and reconstructs only for the predicted class using

   $$\hat{x}_m = \mu_m^{(\hat{y})} + (x_o - \mu_o^{(\hat{y})})B_{\hat{y}}.$$

5. This is more efficient than reconstructing the missing pixels for every class before taking the $\arg\max$. In this implementation, the expensive matrix pseudo-inverse and log-determinant computations are done only once per class, and reconstruction is performed only for the final predicted class.
6. For matrix inversion, I used `lin.pinv(...)` instead of `lin.inv(...)` because the covariance submatrix Σ_{oo} can be singular or nearly singular. I also used `lin.slogdet(...)` for numerically stable log-determinant computation. If `slogdet(...)` returns a non-positive sign, I set the corresponding log-determinant contribution to zero as a numerical safeguard, following the same spirit as the lecture code for degenerate covariance blocks.
7. Finally, the function `truncatePixels(...)` is applied at the end to clip reconstructed pixel values to the valid intensity range $[0, 1]$.

Pseudocode Summary

1. For each class, precompute $\mu_o^{(c)}$, $\mu_m^{(c)}$, $(\Sigma_{oo}^{(c)})^{\dagger}$, B_c, $\bar{\Sigma}_c$, and the constant α_c.
2. For each test image, compute the joint score for every class.
3. Predict the class with the maximum joint score.
4. Reconstruct the missing pixels only for the predicted class.
5. Clip reconstructed values into the valid range $[0, 1]$.

Implementation

```
def precomputeJointTerms(model, mask):
    miss = ~mask
    jointModel = []

    for mu, Sigma, c, p in model:
        mu_o = mu[mask]
        mu_m = mu[miss]

        Soo = Sigma[mask, :][:, mask]
        Som = Sigma[mask, :][:, miss]
        Smo = Sigma[miss, :][:, mask]
        Smm = Sigma[miss, :][:, miss]

        # Use pseudoinverse because Soo may be singular
        SooInv = lin.pinv(Soo)

        sign_oo, logdet_oo = lin.slogdet(Soo)
        if sign_oo <= 0:
            logdet_oo = 0.0

        condCov = Smm - Smo.dot(SooInv).dot(Som)

        sign_c, logdet_c = lin.slogdet(condCov)
        if sign_c <= 0:
            logdet_c = 0.0

        # Conditional mean can be written as:
        # mu_m + (x_o - mu_o) @ B
        B = SooInv.dot(Som)

        const = np.log(p) - 0.5 * logdet_oo - 0.5 * logdet_c

        jointModel.append((c, mu_o, mu_m, SooInv, B, const))

    return jointModel


def predictJointAndReconFast(X, jointModel, C, mask=[]):
    if len(mask) == 0:
        mask = np.ones((X.shape[1],), dtype=bool)

    miss = ~mask
    Xo = X[:, mask]
    n = X.shape[0]

    classScores = np.zeros((n, C))

    # Step 1: compute joint scores for all classes
    for c, mu_o, mu_m, SooInv, B, const in jointModel:
        XoCent = Xo - mu_o
        quad = np.sum((XoCent.dot(SooInv)) * XoCent, axis=1)
        classScores[:, c] = const - 0.5 * quad

    yPred = np.argmax(classScores, axis=1)

    # Step 2: reconstruct only for the predicted class
    XRecon = np.zeros(X.shape)
    XRecon[:, mask] = X[:, mask]

    if np.sum(miss) > 0:
        for c, mu_o, mu_m, SooInv, B, const in jointModel:
            idx = (yPred == c)
            if np.any(idx):
```

```
                XoCent = X[idx][:, mask] - mu_o
                XRecon[idx][:, miss] = mu_m + XoCent.dot(B)

    return yPred, truncatePixels(XRecon)
```

Listing 1: Efficient single-step joint inference code

Final Change in Experiment Code

For the proposed method, I first precompute the class-specific quantities for the given censoring mask, and then run the fast single-step inference routine. The original lecture functions `predictGen(...)` and `reconstructImages(...)` are retained as the two-step baseline for comparison in Part 3.

Lecture two-step baseline:

```
yPred = predictGen(XTestCensorFlat, model, C, mask)
XTestReconFlat = reconstructImages(XTestCensorFlat[:numRows*numCols],
                                   yPred[:numRows*numCols], model,
    mask)
```

Proposed single-step method:

```
jointModel = precomputeJointTerms(model, mask)
yPred, XTestReconFlat = predictJointAndReconFast(XTestCensorFlat,
                                                 jointModel, C, mask)
```

Complexity

Let d_o be the number of observed pixels, d_m the number of missing pixels, C the number of classes, and n the number of test images.

- Precomputation cost:
$$O\big(C(d_o^3 + d_m^3)\big)$$
since each class requires one pseudo-inverse and one conditional covariance computation.
- Score computation over all test points:
$$O(Cnd_o^2)$$
- Reconstruction after prediction:
$$O(nd_od_m)$$
- Hence, the expensive matrix operations are done only once per class, and reconstruction is performed only for the final predicted class.

Conclusion

The training process remains unchanged, while the test-time inference code is modified so that classification and missing-pixel reconstruction are performed jointly in a single step. Compared to a naive implementation that reconstructs every class before taking the $\arg\max$, the above algorithm is more efficient because it precomputes class-specific matrix terms once and reconstructs only for the winning class. The original two-step lecture code is retained as a baseline, and the new joint-inference function is used for comparison in Part 3.

0.1 Part 3: Performance Comparison

Using our modified inference architecture, we evaluated the performance of both single-step and two-step prediction models across varying degrees of censoring. The `censorImages` function was bound to remove central portions of lengths corresponding to 51% (4:24), 32% (6:22), 18% (8:20), 8% (10:18), and 2% (12:16) missing pixels.

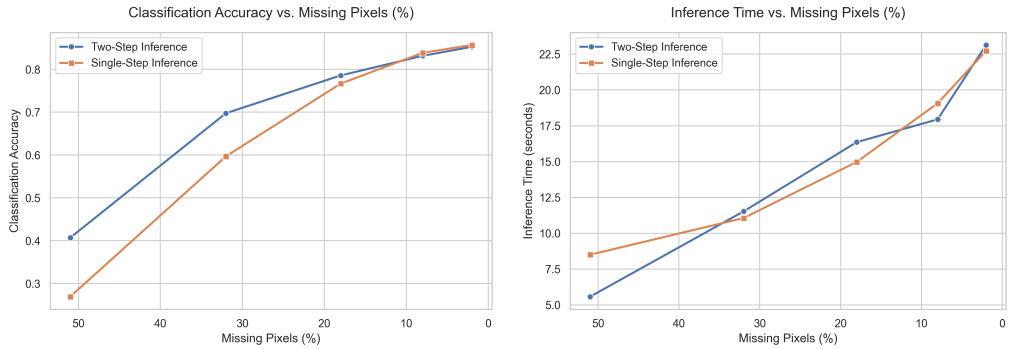

Figure 1: Left: Classification Accuracy vs. Missing Pixels (%). Right: Inference Time vs. Missing Pixels (%). Generated via seaborn formatting.

Classification Performance Analysis:
As censoring becomes less aggressive (fewer missing pixels), both methods naturally show improved classification performance. However, the **two-step prediction approach consistently achieves strictly higher accuracy**. The pronounced drop in single-step accuracy under moderate to high censoring (e.g., 51% missing) largely stems from the added penalty term ($-\frac{1}{2}\log|\bar{\Sigma}_c|$). Due to severe structural rank deficiencies in static MNIST background pixels, this marginalized covariance determinant becomes numerically singular, leading to structural handicap in the joint MAP estimation.

Inference Time Analysis:
A pronounced counter-intuitive behavior dictates inference time: it scales inverse to missing proportions. The time is heavily dominated by the required size of the pseudo-inverse operation applied to the remaining observed pixels (Σ_{oo}). When censoring becomes less aggressive (e.g., only 2% missing), almost all parameters represent "observed" variables, drastically ballooning the $d_o \times d_o$ dimension of Σ_{oo}. Because the $O(d_o^3)$ computationally expensive pseudo-inverse fundamentally dictates the clock time, the supplementary matrix multiplications applied internally by the single-step overhead ($\Sigma_{mo}S^{-1}\Sigma_{om}$) contribute only a marginal relative delay, causing both algorithms to scale similarly.

Reconstruction Quality Variation:
It is crucial to structurally establish that the latent reconstruction step formally defined as $\hat{x}_m = \arg\max_v P[v \mid x_o, y = \hat{y}]$ is mathematically identical across both models. As a consequence, providing both methods output the same predicted class label, **both methods must produce identical reconstructed images**. Differences in visual reconstruction quality rely entirely on discrepancies in downstream classification accuracy:

- **Aggressive Censoring (51%, 32%):** The two-step method correctly classifies a substantially higher proportion of digits compared to the single-step method. Therefore, the two-step approach yields consistently higher visual reconstruction accuracy since it utilizes the proper base digit structure as its predictive prior rather than sampling expected conditional mean values from random or incorrect digit classes.

- **Minor Censoring (8%, 2%):** The label assignments of both methods begin to rapidly converge. Both methods correctly identify the true latent labels for the vast majority of test instances, establishing virtually identical high-quality reconstructions.

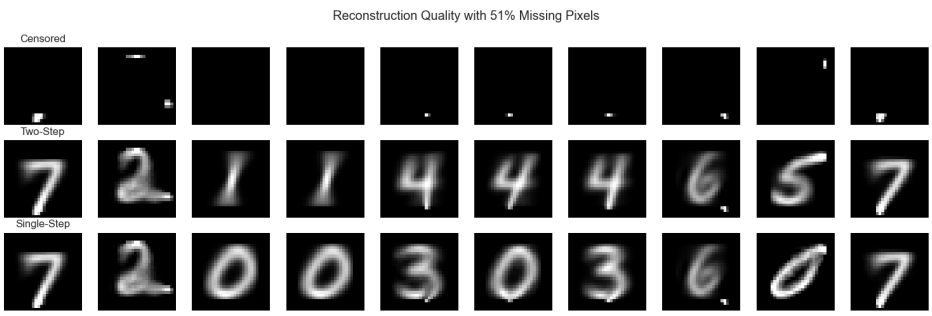

Figure 2: Visual reconstruction comparisons at 51% aggressive censoring.

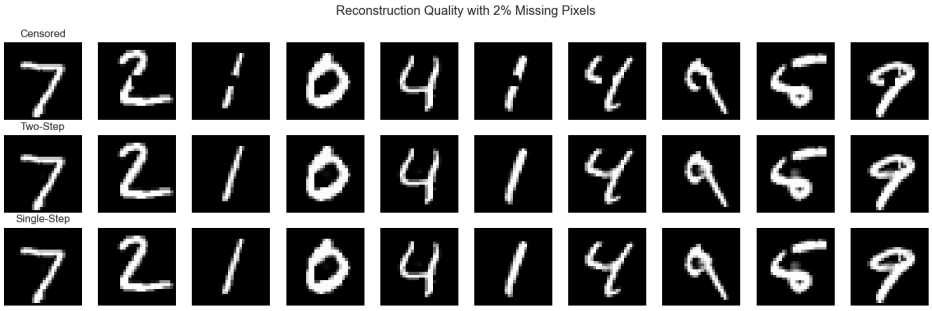

Figure 3: Visual reconstruction comparisons at 2% minor censoring.